Assignment No: 6

Title: Link JS file and apply basic events, array functions.

Aim: To link an external JavaScript file to an HTML document and to apply basic DOM events and the standard higher-order array functions map, filter, reduce, forEach, find, some and sort.

Objectives:

1. To learn the different ways of linking a JavaScript file to a webpage.
2. To understand the difference between a classic script, a deferred script and an ES module.
3. To attach event handlers using HTML attributes and using addEventListener.
4. To apply the array higher-order functions map, filter, reduce, forEach, find and some.
5. To sort an array of objects using a custom comparator function.
6. To understand that higher-order array functions return a new array instead of mutating the original one.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- Node.js (for running the project's module bundler)

Theory:

1. Linking a JavaScript File

JavaScript can be included in a webpage in three ways: inline inside an HTML attribute, internally inside a <script> block, or externally in a separate .js file linked with the src attribute. The external method is preferred because it separates behaviour from structure and allows the browser to cache the file.

Form	Meaning
<script src="app.js"></script>	Classic script, downloaded and executed immediately, blocking parsing.
<script src="app.js" defer></script>	Downloaded in parallel, executed after the HTML is parsed.
<script type="module" src="app.js"></script>	ES module: supports import/export and is deferred by default.

2. ES Modules - export and import

An ES module exposes values with export and consumes them with import. The browser loads the dependency graph itself, so one <script type="module"> entry point is enough for an entire application. This is how the LipiSutra project links all of its JavaScript.

Example:

```
// tts.js
export function speakText(text) { ... }

// PublicView.jsx
import { speakText } from "../services/tts";
```

3. Basic Events

An event is a signal that something has happened in the browser. A handler is a function registered to run when that signal fires.

Event	Fires when
click	the element is clicked
change	the value of a form control is committed
input	the value of a form control changes on every keystroke
submit	a form is submitted
keydown / keyup	a key is pressed or released
dragover / drop	a dragged item is moved over or dropped on a target
load / DOMContentLoaded	the page or the DOM has finished loading

Registering a handler:

```
// HTML attribute (not recommended)
<button onclick="decode()">Decode</button>

// addEventListener (recommended, allows several handlers)
btn.addEventListener('click', decode);

// React synthetic event
<button onClick={decode}>Decode</button>
```

4. Higher-Order Array Functions

Function	Returns	Purpose
map()	new array	transforms every element
filter()	new array	keeps elements that pass a test
reduce()	single value	folds the array into one accumulated value
forEach()	undefined	runs a function for its side effect only

find()	one element	the first element passing a test
some() / every()	boolean	whether any / all elements pass a test
sort()	the same array	orders the array using a comparator
join()	string	concatenates the elements with a separator

map, filter and reduce are pure: they do not modify the original array. sort is an exception - it sorts in place and returns the same array reference, so a copy should be taken first when the original order matters.

Course Project Implementation:

Course-Project Mapping: Available in the course project. Submit src/services/vision.js - it is an external JavaScript module linked by ES import, and its entire body is a chain of forEach, map, join and push over the nested Google Vision response. Submit src/components/MuseumView.jsx alongside it for map, slice and reduce-style accumulation over the Firestore snapshot, and src/components/Navbar.jsx for map combined with an onClick event handler.

Concept	File	Evidence
External JS file linked	src/services/vision.js	imported by UploadSection.jsx; entry point index.html -> main.jsx
forEach / map / join / push	src/services/vision.js	flattening pages > blocks > paragraphs > words
map + onClick event	src/components/Navbar.jsx	roles.map(r => <button onClick=.../>)
map, slice, accumulation	src/components/MuseumView.jsx	snapshot.docs.map(...), docs.slice(0, 5), confSum accumulation
sort with comparator	src/components/HistorianView.jsx	pending.sort((a, b) => parseAccuracy(a) - parseAccuracy(b))
split / filter / replace	src/services/tts.js, HistorianView.jsx	tag stripping and word-by-word diffing

Program:

Project file - src/services/vision.js (external module, array chain)

```
const GOOGLE_KEY = import.meta.env.VITE_VISION_KEY;

export async function getConfidenceScores(base64Image) {
  const response = await fetch(
    `https://vision.googleapis.com/v1/images:annotate?key=${GOOGLE_KEY}`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({
        requests: [{
          image: { content: base64Image },
          features: [{ type: "DOCUMENT_TEXT_DETECTION" }]
        }]
      })
    }
  );

  const data  = await response.json();
  const words = [];
  const pages = data.responses[0]?.fullTextAnnotation?.pages || [];

  // forEach nested four levels deep, map + join to rebuild each word
  pages.forEach(page =>
    page.blocks?.forEach(block =>
      block.paragraphs?.forEach(para =>
        para.words?.forEach(word => {
          const text = word.symbols?.map(s => s.text).join("") || "";
          words.push({ word: text, confidence: word.confidence || 0 });
        })
      )
    )
  );

  return words;
}
```

Project file - src/components/Navbar.jsx (map + click event)

```
const roles = [
  { id: "public",    label: "Public / Student" },
  { id: "historian", label: "Historian" },
  { id: "museum",    label: "Museum Admin" }
];

{roles.map(r => (
  <button
    key={r.id}
    onClick={() => onViewChange(r.id)}          // click event handler
    className="px-5 py-2 text-xs rounded-full border"
  >
    {r.label}
  </button>
))}
```

Project file - src/components/HistorianView.jsx (sort with a comparator)

```
const unsubscribe = onSnapshot(q, (snapshot) => {
  const pending = [];
  snapshot.docs.forEach(docSnap => {
    const data = docSnap.data();
    if (data.status === 'pending') pending.push({ id:
docSnap.id, ...data.ai_output });
  });

  // lowest confidence first -> the documents a historian should review soonest
  pending.sort((a, b) => {
    const accA = parseAccuracy(a);
    const accB = parseAccuracy(b);
    return accA - accB;
  });

  setDocuments(pending);
});
```

Supplementary standalone demonstration - assignment06.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>LipiSutra - Events and Array Functions</title>
</head>
<body>
  <h1>LipiSutra Archive Console</h1>

  <button id="btn-all">Show All</button>
  <button id="btn-verified">Verified Only</button>
  <button id="btn-stats">Compute Statistics</button>
  <input id="search" type="text" placeholder="Filter by script, then press Enter">

  <ul id="list"></ul>
  <p id="stats"></p>

  <!-- EXTERNAL JAVASCRIPT FILE LINKED HERE -->
  <script src="assignment06.js" defer></script>
</body>
</html>
```

Supplementary standalone demonstration - assignment06.js

```
"use strict";

// ---------- sample data : the LipiSutra archive ----------
const manuscripts = [
  { id: "M-101", script: "Modi",        era: "1712 CE", confidence: 0.92, status:
"verified" },
  { id: "M-102", script: "Devanagari",  era: "1588 CE", confidence: 0.87, status:
"verified" },
  { id: "M-103", script: "Brahmi",      era: "230 BCE", confidence: 0.71, status:
"pending"  },
  { id: "M-104", script: "Sharada",     era: "1104 CE", confidence: 0.80, status:
"pending"  },
  { id: "M-105", script: "Modi",        era: "1680 CE", confidence: 0.95, status:
"verified" }
];

const listEl   = document.getElementById("list");
const statsEl  = document.getElementById("stats");
const searchEl = document.getElementById("search");

// ---------- render helper : map() + join() ----------
function render(rows) {
  if (rows.length === 0) {
    listEl.innerHTML = "<li>No manuscripts match.</li>";
    return;
  }
  listEl.innerHTML = rows
    .map(m => `<li><b>${m.id}</b> - ${m.script} (${m.era}) - `
            + `${Math.round(m.confidence * 100)}% - ${m.status}</li>`)
    .join("");
}

// ---------- EVENT 1 : click ----------
document.getElementById("btn-all")
  .addEventListener("click", () => render(manuscripts));

// ---------- EVENT 2 : click + filter() ----------
document.getElementById("btn-verified").addEventListener("click", () => {
  const verified = manuscripts.filter(m => m.status === "verified");
  render(verified);
});
```

```
// ---------- EVENT 3 : click + reduce() / find() / some() ----------
document.getElementById("btn-stats").addEventListener("click", () => {
  const total = manuscripts.reduce((sum, m) => sum + m.confidence, 0);
  const avg   = (total / manuscripts.length) * 100;

  const best    = manuscripts.find(m => m.confidence >= 0.95);
  const anyLow  = manuscripts.some(m => m.confidence < 0.75);

  // sort on a COPY so the original order is preserved
  const ranked = [...manuscripts].sort((a, b) => b.confidence - a.confidence);

  statsEl.textContent =
    `Average confidence ${avg.toFixed(1)}% | ` +
    `best = ${best ? best.id : "none"} | ` +
    `needs review = ${anyLow ? "yes" : "no"} | ` +
    `ranked = ${ranked.map(m => m.id).join(" > ")}`;
});

// ---------- EVENT 4 : keydown on a text input ----------
searchEl.addEventListener("keydown", (e) => {
  if (e.key !== "Enter") return;
  const term = e.target.value.trim().toLowerCase();
  const hits = manuscripts.filter(m => m.script.toLowerCase().includes(term));
  render(hits);
});

// ---------- EVENT 5 : page load, with forEach() for a side effect ----------
window.addEventListener("DOMContentLoaded", () => {
  manuscripts.forEach(m => console.log("Loaded", m.id, m.script));
  render(manuscripts);
});
```

Output:

- On page load the DOMContentLoaded handler logs all five manuscripts to the console and renders the full list.
- Clicking "Verified Only" uses filter() and re-renders three rows; the original array is unchanged, as clicking "Show All" proves.
- Clicking "Compute Statistics" prints: Average confidence 85.0% | best = M-105 | needs review = yes | ranked = M-105 > M-101 > M-102 > M-104 > M-103.
- Typing "modi" in the search box and pressing Enter filters the list down to M-101 and M-105.
- In the running LipiSutra application the same techniques produce the historian queue: the lowest-confidence documents are sorted to the top of the review list.

Conclusion:

In this assignment, we linked an external JavaScript file to a webpage using both a classic deferred script and ES modules, attached handlers for the click, keydown and DOMContentLoaded events using addEventListener, and applied map, filter, reduce, forEach, find, some, sort and join. We also traced the same patterns in the LipiSutra project, where vision.js flattens a deeply nested API response with forEach/map/join and HistorianView.jsx sorts the review queue with a custom comparator.

References:

7. Mozilla Developer Network (MDN) Web Docs - JavaScript Guide, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
8. W3Schools - JavaScript Tutorial, https://www.w3schools.com/js/
9. ECMAScript Language Specification, https://tc39.es/ecma262/
10. MDN Web Docs - Array, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array